FRED TALKS

10th August 2026

CONTENTS

Scaling Managed Agents: Decoupling the brain from the hands

ANTHROPIC ENGINEERING BLOG · 24 APR 2026 · SOURCE

Get started with Claude Managed Agents by following our docs.

A running topic on the Engineering Blog is how to build effective agents and design harnesses for long-running work. A common thread across this work is that harnesses encode assumptions about what Claude can't do on its own. However, those assumptions need to be frequently questioned because they can go stale as models improve.

As just one example, in prior work we found that Claude Sonnet 4.5 would wrap up tasks prematurely as it sensed its context limit approaching—a behavior sometimes called "context anxiety." We addressed this by adding context resets to the harness. But when we used the same harness on Claude Opus 4.5, we found that the behavior was gone. The resets had become dead weight.

We expect harnesses to continue evolving. So we built Managed Agents: a hosted service in the Claude Platform that runs long-horizon agents on your behalf through a small set of interfaces meant to outlast any particular implementation—including the ones we run today.

Building Managed Agents meant solving an old problem in computing: how to design a system for "programs as yet unthought of." Decades ago, operating systems solved this problem by virtualizing hardware into abstractions—*process*, *file*—general enough for programs that didn't exist yet. The abstractions outlasted the hardware. The `read()` command is agnostic as to whether it's accessing a disk pack from the 1970s or a modern SSD. The abstractions on top stayed stable while the implementations underneath changed freely.

Managed Agents follow the same pattern. We virtualized the components of an agent: a session (the append-only log of everything that happened), a harness (the loop that calls Claude and routes Claude's tool calls to the relevant infrastructure), and a sandbox (an execution environment where Claude can run code and edit files). This allows the implementation of each to be swapped without disturbing the others. We're opinionated about the shape of these interfaces, not about what runs behind them.

Control the ideas, not the code

ANTIREZ.COM · 18 JUL 2026 · SOURCE

Look at the past history of this blog. There are many blog posts abou

So mine is a trick. People feel more and more programming is complete

This is why yesterday, on X, I said that I believe many programmers a

1. You can now generate a lot of code, even *not* accounting for the

2. LLMs are very good at writing locally optimal code, and are worse

3. The working day is 8 hours. If you read the code, it is a tradeoff

Controlling the ideas. Do you remember this phrasing from the Mythica

Matteo Collina yesterday asked me, in reply to my tweet: but didn't y

Fable and GPT 5.6 reviews to the sorted sets memory saving are going

:

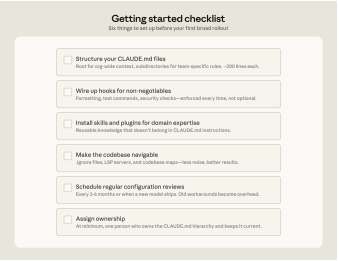
Bottoms-up adoption generates enthusiasm but can fragment without someone to centralize what works. You need to have an individual or a team assemble and evangelize the right Claude Code conventions (such as a standardized CLAUDE.md hierarchy or a curated set of skills and plugins). Without that work, knowledge will stay tribal and adoption will plateau.

In large organizations, especially those in regulated industries, governance questions come up early, such as: who controls which skills and plugins are available, how do you prevent thousands of engineers from independently rebuilding the same thing, how do you make sure AI-generated code goes through the same review process as human-generated code? To address these early on, we suggest starting with a defined set of approved skills, required code review processes, and limited initial access, and expand as confidence builds.

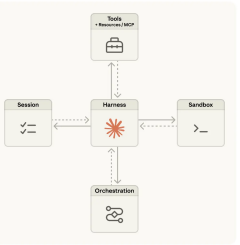
We've observed the smoothest deployments at organizations that establish cross-functional working groups early by bringing together engineering, information security, and governance representatives to define requirements together and build a rollout roadmap.

Applying these patterns to your organization

Claude Code is designed around conventional software engineering environments where engineers are the primary codebase contributors, the repo uses Git, and code follows standard directory structures. Most large codebases fit this mold, but non-traditional setups such as game engines with large binary assets, environments with unconventional version control, or non-engineers contributing to the codebase require additional configuration work. Our guidance assumes a conventional setup and the patterns we've described have worked across many of our customers. Any remaining complexity requires judgment specific to your codebase, tooling, and organization. That's where Anthropic's Applied AI team works directly with engineering teams to translate these patterns into your organization's specific requirements.



Acknowledgements: *Special thanks to Alon Krifcher, Charmaine Lee, Chris Concannon, Harsh Patel, Henrique Savelli, Jason Schwartz, Jonah Dueck and Kirby Kohlmorgen from Anthropic's Applied AI team for sharing their experience deploying Claude Code at scale, and to Amit Navindgi at Zoox for providing feedback on this article.*



We started by placing all agent components into a single container, which meant the session, agent harness, and sandbox all shared an environment. There were benefits to this approach, including that file edits are direct syscalls, and there were no service boundaries to design.

But by coupling everything into one container, we ran into an old infrastructure problem: we'd adopted a *pet*. In the pets-vs-cattle analogy, a pet is a named, hand-tended individual you can't afford to lose, while cattle are interchangeable. In our case, the server became that pet; if a container failed, the session was lost. If a container was unresponsive, we had to nurse it back to health.

Nursing containers meant debugging unresponsive stuck sessions. Our only window in was the WebSocket event stream, but that couldn't tell us *where* failures arose, which meant that a bug in the harness, a packet drop in the event stream, or a container going offline all presented the same. To figure out what went wrong, an engineer had to open a shell inside the container, but because that container often also held user data, that approach essentially meant we lacked the ability to debug.

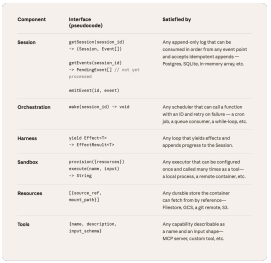
A second issue was that the harness assumed that whatever Claude worked on lived in the container with it. When customers asked us to connect Claude to their virtual private cloud, they had to either peer their network with ours, or run our harness in their own environment. An assumption baked into the harness became a problem when we wanted to connect it to different infrastructure.

The solution we arrived at was to decouple what we thought of as the "brain" (Claude and its harness) from both the "hands" (sandboxes and tools that perform actions) and the "session" (the log of session events). Each became an interface that made few assumptions about the others, and each could fail or be replaced independently.

The harness leaves the container. Decoupling the brain from the hands meant the harness no longer lived inside the container. It called the container the way it called any other tool: `execute(name, input) → string`. The container became cattle. If the container died,

the harness caught the failure as a tool-call error and passed it back to Claude. If Claude decided to retry, a new container could be reinitialized with a standard recipe: `provision({resources})`. We no longer had to nurse failed containers back to health.

Recovering from harness failure. The harness also became cattle. Because the session log sits outside the harness, nothing in the harness needs to survive a crash. When one fails, a new one can be rebooted with `wake(sessionId)`, use `getSession(id)` to get back the event log, and resume from the last event. During the agent loop, the harness writes to the session with `emitEvent(id, event)` in order to keep a durable record of events.



Component	Interface (pseudocode)	Satisfied by

Session	`getSession(session_id) -> [Session, Event[]]`
	`getEvents(session_id) -> PendingEvent[] // not yet processed`
	`emitEvent(id, event)`
		Any append-only log that can be consumed in order from any event point and accepts idempotent appends—Postgres, SQLite, in-memory array, etc.

Orchestration	`wake(session_id) -> void`	Any scheduler that can call a function with an ID and retry on failure—a cron job, a queue consumer, a while-loop, etc.
Harness	`yield Effect<T> -> EffectResult<T>`	Any loop that yields effects and appends progress to the Session.
Sandbox	`provision([resources]) execute(name, input) -> string`	Any executor that can be configured once and called many times as a tool—a local process, a remote container, etc.
Resources	`[{source_ref, mount_path}]`	Any durable store the container can fetch from by reference—Filestore, GCS, a git remote, S3.
Tools	`{name, description, input_schema}`	Any capability describable as a name and an input shape—MCP server, custom tool, etc.

The security boundary. In the coupled design, any untrusted code that Claude generated was run in the same container as credentials—so a prompt injection only had to convince Claude to read its own environment. Once an attacker has those tokens, they can spawn fresh, unrestricted sessions and delegate work to them. Narrow scoping is an obvious mitigation, but this encodes an assumption about what Claude can't do with a limited token—and Claude is getting increasingly smart. The structural fix was to make sure the tokens are never reachable from the sandbox where Claude's generated code runs.

We used two patterns to ensure this. Auth can be bundled with a resource or held in a vault outside the sandbox. For Git, we use each repository's access token to clone the repo during sandbox initialization and wire it into the local git remote. Git `push` and `pull` work from inside the sandbox without the agent ever handling the token itself. For custom tools, we support MCP and store OAuth tokens in a secure vault. Claude calls MCP tools via a dedicated proxy; this proxy takes in a token associated with the session. The proxy can then fetch the corresponding credentials from the vault and make the call to the external service. The harness is never made aware of any credentials.

The session is not Claude's context window

Long-horizon tasks often exceed the length of Claude's context window, and the standard ways to address this all involve irreversible decisions about what to keep. We've explored these techniques in prior work on context engineering. For example, compaction lets Claude save a

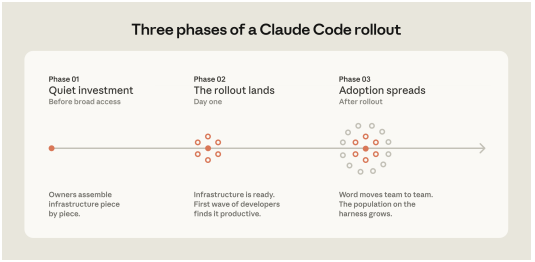
Skills and hooks built to compensate for specific model limitations, whether in the model's reasoning or in Claude Code's own tooling, become overhead once those limitations no longer exist. A hook that intercepted file writes to enforce p4 edit in a Perforce codebase, for example, became redundant once Claude Code added native Perforce mode.

Teams should expect to do a meaningful configuration review every three to six months, but it's also worth doing one whenever performance feels like it's plateaued after major model releases.

Assigning ownership for Claude Code management and adoption

Technical configuration alone doesn't drive adoption. The organizations that got it right invested in the organizational layer, too.

The rollouts that spread fastest had a dedicated infrastructure investment before broad access. A small team, sometimes even just one person, wired up the tooling so Claude already fit developer workflows when they first touched it. At one company, a couple of engineers built a suite of plugins and MCPs that were available on day one. At another, an entire team focused on managing AI coding tools had the infrastructure in place before the rollout began. In both cases, developers' first experience was productive rather than frustrating, and adoption spread from there.



The teams doing this work today tend to sit under developer experience or developer productivity, which is typically the function responsible for onboarding new engineers and building developer tooling. An emerging role in several organizations is an agent manager: a hybrid PM/engineer function dedicated to managing the Claude Code ecosystem. For organizations without a dedicated team, the minimum viable version is a DRI: one person with ownership over the Claude Code configuration, the authority to make calls on settings, permissions policy, the plugin marketplace, and CLAUDE.md conventions, and the responsibility to keep them current.

- **Using `.ignore` files to exclude generated files, build artifacts, and third-party code.** Committing `permissions.deny` rules in `.claude/settings.json` means the exclusions are version-controlled, so every developer on the team gets the same noise reduction without configuring it themselves. In some codebases, generated files are themselves the subject of development work. Developers who work on code generators can override project-level exclusions in their local settings without affecting the rest of the team.

- **Building codebase maps when the directory structure doesn't do the work.** For organizations where code isn't consolidated in a conventional directory structure, a lightweight markdown file at the repo root listing each top-level folder with a one-line description of what lives there gives Claude a table of contents it can scan before opening files. For codebases with hundreds of top-level folders, this works best as a layered approach: the root file describes only the highest-level structure, and subdirectory CLAUDE.md files provide the next level of detail, loading on demand as Claude moves through the tree. For simpler cases, @-mentioning the specific files or directories Claude should reference can do the same job.

- **Running LSP servers so Claude searches by symbol, not by string.** Grep for a common function name in a large codebase returns thousands of matches and Claude burns context opening files to figure out which matters. LSP returns only the references that point to the same symbol, so the filtering happens before Claude reads anything. Setting this up requires installing a code intelligence plugin for your language and the corresponding language server binary; the Claude Code documentation covers the available plugins and troubleshooting.

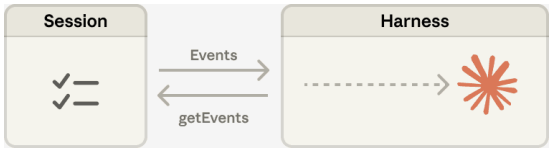
One caveat: there are edge cases where even the hierarchical CLAUDE.md approach breaks down, for example codebases with hundreds of thousands of folders and millions of files, or legacy systems on non-git version control. We will address their challenges in future installments of this series.

Actively maintaining CLAUDE.md files as model intelligence evolves

As models evolve, instructions written for your current model can work against a future one. CLAUDE.md files that guided Claude through patterns it used to struggle with may either become unnecessary or actively constraining when the next model ships. For example, a CLAUDE.md rule that tells Claude to break every refactor into single-file changes may have helped an earlier model stay on track but would prevent a newer one from making coordinated cross-file edits it handles well.

summary of its context window and the memory tool lets Claude write context to files, enabling learning across sessions. This can be paired with context trimming, which selectively removes tokens such as old tool results or thinking blocks.

But irreversible decisions to selectively retain or discard context can lead to failures. It is difficult to know which tokens the future turns will need. If messages are transformed by a compaction step, the harness removes compacted messages from Claude's context window, and these are recoverable only if they are stored. Prior work has explored ways to address this by storing context as an object that lives *outside* the context window. For example, context can be an object in a REPL that the LLM programmatically accesses by writing code to filter or slice it.



In Managed Agents, the session provides this same benefit, serving as a context object that lives outside Claude's context window. But rather than be stored within the sandbox or REPL, context is durably stored in the session log. The interface, `getEvents()`, allows the brain to interrogate context by selecting positional slices of the event stream. The interface can be used flexibly, allowing the brain to pick up from wherever it last stopped reading, rewinding a few events before a specific moment to see the lead up, or rereading context before a specific action.

Any fetched events can also be transformed in the harness before being passed to Claude's context window. These transformations can be whatever the harness encodes, including context organization to achieve a high prompt cache hit rate and context engineering. We separated the concerns of recoverable context storage in the session and arbitrary context management in the harness because we can't predict what specific context engineering will be required in future models. The interfaces push that context management into the harness, and only guarantee that the session is durable and available for interrogation.

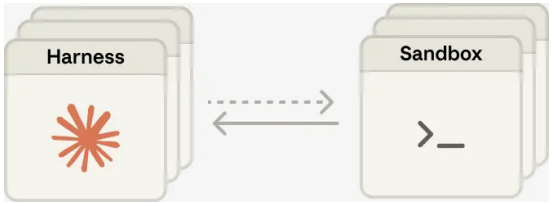
Many brains. Decoupling the brain from the hands solved one of our earliest customer complaints. When teams wanted Claude to work against resources in their own VPC, the only path was to peer their network with ours, because the container holding the harness assumed every resource sat next to it. Once the harness was no longer in the container, that assumption went away. The same change had a performance payoff. When we initially put the brain in a container, it meant that many brains required as many containers. For each brain, no inference could happen until that container was provisioned; every session paid the full container setup cost up front. Every session, even ones that would never touch the sandbox, had to clone the repo, boot the process, fetch pending events from our servers.

That dead time is expressed in time-to-first-token (TTFT), which measures how long a session waits between accepting work and producing its first response token. TTFT is the latency the user most acutely *feels*.

Decoupling the brain from the hands means that containers are provisioned by the brain via a tool call `(execute(name, input) → string)` only if they are needed. So a session that didn't need a container right away didn't wait for one. Inference could start as soon as the orchestration layer pulled pending events from the session log. Using this architecture, our p50 TTFT dropped roughly 60% and p95 dropped over 90%. Scaling to many brains just meant starting many stateless harnesses, and connecting them to hands only if needed.

Many hands. We also wanted the ability to connect each brain to many hands. In practice, this means Claude must reason about many execution environments and decide where to send work —a harder cognitive task than operating in a single shell. We started with the brain in a single container because earlier models weren't capable of this. As intelligence scaled, the single container became the limitation instead: when that container failed, we lost state for every hand that the brain was reaching into.

Decoupling the brain from the hands makes each hand a tool, `execute(name, input) → string`: a name and input go in, and a string is returned. That interface supports any custom tool, any MCP server, and our own tools. The harness doesn't know whether the sandbox is a container, a phone, or a Pokémon emulator. And because no hand is coupled to any brain, brains can pass hands to one another.



The challenge we faced is an old one: how to design a system for "programs as yet unthought of." Operating systems have lasted decades by virtualizing the hardware into abstractions general enough for programs that didn't exist yet. With Managed Agents, we aimed to design a system that accommodates future harnesses, sandboxes, or other components around Claude.

Managed Agents is a meta-harness in the same spirit, unopinionated about the *specific* harness that Claude will need in the future. Rather, it is a system with general interfaces that allow many different harnesses. For example, Claude Code is an excellent harness that we use widely across tasks. We've also shown that task-specific agent harnesses excel in narrow domains. Managed Agents can accommodate any of these, matching Claude's intelligence over time.

Subagents*	Separate Claude instances for specific tasks	When invoked	Splitting exploration from editing, parallel work	Running exploration and editing in the same session

*LSP is accessed through the plugin layer. Subagents are a delegation capability rather than a configured extension point.

Three configuration patterns from successful deployments

How you configure Claude Code for a large codebase depends heavily on how that codebase is structured. Still, three patterns appeared consistently across the deployments we observed.

Making the codebase navigable at scale

Claude's ability to help in a large codebase is bounded by its ability to find the right context. Too much context loaded into every session degrades performance, while too little context leaves Claude to navigate blind. The most effective deployments invest upfront in making the codebase legible to Claude. A few patterns appear consistently:

- **Keeping CLAUDE.md files lean and layered.** Claude loads them additively as it moves through the codebase: root file for the big picture, subdirectory files for local conventions. The root file should be pointers and critical gotchas only; everything else drifts into noise.

- **Initializing in subdirectories, not at the repo root.** Claude works best when it's scoped to the part of the codebase that's actually relevant to the task. In monorepos, this can feel counterintuitive because tooling often assumes root access, but Claude automatically walks up the directory tree and loads every CLAUDE.md file it finds along the way, so root-level context is never lost.

- **Scoping test and lint commands per subdirectory.** Running the full suite when Claude changed one service causes timeouts and wastes context on irrelevant output. CLAUDE.md files at the subdirectory level should specify the commands that apply to that part of the codebase. This works well for service-oriented codebases where each directory has its own test and build commands. In compiled-language monorepos with deep cross-directory dependencies, per-subdirectory scoping is harder to achieve and may require project-specific build configurations.

Component	What it is	When it loads	Best for	Common confusion
CLAUDE.md	Context file Claude reads automatically	Every session	Project-specific conventions, codebase knowledge	Using it for reusable expertise that belongs in a skill
Hooks	Scripts that run at key moments	Triggered by events	Automating consistent behavior, capturing session learnings	Using prompts for things that should run automatically
Skills	Packaged instructions for specific task types	On demand, when relevant	Reusable expertise across sessions and projects	Loading everything into CLAUDE.md instead
Plugins	Bundled skills, hooks, MCP configs	Always available once configured	Distributing a working setup across the org	Letting good setups stay tribal
Language server protocol (LSP)*	Real-time code intelligence via language specific servers	Always available once configured	Symbol-level navigation and automatic error detection in typed languages	Assuming that it's automatic
MCP servers	Connections to external tools and data	Always available once configured	Giving Claude access to internal tools it can't otherwise reach	Building MCP connections before the basics are working

Meta-harness design means being opinionated about the interfaces around Claude: we expect that Claude will need the ability to manipulate state (the session) and perform computation (the sandbox). We also expect that Claude will require the ability to scale to many brains and many hands. We designed the interfaces so that these can be run reliably and securely over long time horizons. But we make no assumptions about the number or location of brains or hands that Claude will need.

Written by Lance Martin, Gabe Cemaj and Michael Cohen. Special thanks to the Agents API team and Jake Eaton for their contributions.

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · SOURCE

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments…). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-restatement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs… but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding

The table below summarizes what each component does, when it loads, and the most common mistakes we see with each:

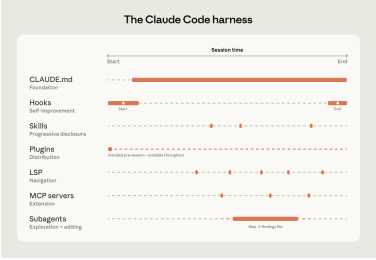
Plugins distribute what works. One challenge with large codebases is that *good* setups can stay tribal. A plugin bundles skills, hooks, and MCP configurations into a single installable package, so when a new engineer installs that plugin on day one, they will immediately have the same context and capabilities as those who have been using Claude already. Plugin updates can be distributed across the organization through managed marketplaces.

For example, a large retail organization we work with built a skill connecting Claude to their internal analytics platform so that business analysts could pull performance data without leaving their workflow. They distributed it as a plugin before the broad rollout to the business.

Language server protocol (LSP) integrations give Claude the same navigation a developer has in their IDE. Most large-codebase IDEs already have an LSP running, powering "go to definition" and "find all references." Surfacing this to Claude gives it symbol-level precision: it can follow a function call to its definition, trace references across files, and distinguish between identically named functions in different languages. Without it, Claude pattern-matches on text and can land on the wrong symbol. One enterprise software company we worked with deployed LSP integrations org-wide before their Claude Code rollout, specifically to make C and C++ navigation reliable at scale. For multi-language codebases, this is one of the highest-value investments.

MCP servers extend everything. MCP servers are how Claude connects to internal tools, data sources, and APIs that it can't otherwise reach. The most sophisticated teams built MCP servers exposing structured search as a tool Claude can call directly. Others connect Claude to internal documentation, ticketing systems, or analytics platforms.

Subagents split exploration from editing. A subagent is an isolated Claude instance with its own context window that takes a task, does the work, and returns only the final result to the parent. Once the harness is in place, some teams spin up a read-only subagent to map a subsystem and write findings to a file, then have the main agent edit with the full picture.



Claude Code's extension layer at a glance.

doc saying "NEVER TOUCH STATE IN FOOBARDB DIRECTLY!". You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool's existence, or maybe it just preferred FooBarDB's in-distribution API to your esoteric CLI tool. You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name "delete-everything". Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

In recent work called LogAct, my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases (Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!

How Claude Code navigates large codebases

CLAUDE · 18 MAY 2026 · SOURCE

Claude Code is running in production across multi-million-line monorepos, decades-old legacy systems, distributed architectures spanning dozens of repositories, and at organizations with thousands of developers. These environments present challenges that smaller, simpler codebases don’t, whether that’s build commands that differ across every subdirectory or legacy code spread across folders with no shared root.

This article covers the patterns we've observed that have led to successful adoption of Claude Code at scale. We use “large codebase” to refer to a wide range of deployments: monorepos with millions of lines, legacy systems built over decades, dozens of microservices across separate repositories, or any combination of the above. That also includes codebases running on languages that teams don't always associate with AI coding tools, such as C, C++, C#, Java, PHP. (Claude Code performs better than most teams expect it to in those cases, particularly as of recent model releases.) While every large codebase deployment is shaped by its specific version control, team structure, and accumulated conventions, the patterns here generalize across them and are a good starting point for teams considering adopting Claude Code.

Claude Code navigates a codebase the way a software engineer would: it traverses the file system, reads files, uses grep to find exactly what it needs, and follows references across the codebase. It operates locally on the developer’s machine and doesn’t require a codebase index to be built, maintained, or uploaded to a server.

RAG-powered AI coding tools work by embedding the entire codebase and retrieving relevant chunks at query time. At large scale, those systems can fail because embedding pipelines can’t keep up with active engineering teams. By the time a developer queries the index, it reflects the codebase as it previously existed weeks, days, or even hours before. Retrieval then returns a function the team renamed two weeks ago, or references a module that was deleted in the last sprint, with no indication that either is out of date.

Agentic search avoids those failure modes. There's no embedding pipeline or centralized index to maintain as thousands of engineers commit new code. Each developer's instance works from the live codebase.

But the approach has a tradeoff: it works best when Claude has enough starting context to know where to look. This means the quality of Claude's navigation is shaped by how well the codebase is set up, layering context with CLAUDE.md files and skills. If you ask it to find all instances of a vague pattern across a billion-line codebase, you’ll hit context-window limits before the work begins. Teams that invest in codebase setup see better results.

The harness matters as much as the model

One of the most common misconceptions about Claude Code is that its capabilities are solely defined by the model used. Teams focus on a model’s benchmarks and how it performs on test tasks. In practice, the ecosystem built around the model—the harness—determines how Claude Code performs more than the model alone.

The harness is built from five extension points—CLAUDE.md files, hooks, skills, plugins, and MCP servers—each serving a different function. The order in which teams build them matters, as each layer builds on what came before. Two additional capabilities, LSP integrations and subagents, round out the setup. Below, we explain what each of these components and capabilities do:

CLAUDE.md files come first. These are context files that Claude reads automatically at the start of every session: root file for the big picture, subdirectory files for local conventions. They give Claude the codebase knowledge it needs to do anything well. Because they load in every session regardless of the task, keeping them focused on what applies broadly will prevent them from becoming a drag on performance.

Hooks make the setup self-improving. Most teams think of hooks as scripts that prevent Claude from doing something wrong, but their more valuable use is continuous improvement. A stop hook can reflect on what happened during a session and propose CLAUDE.md updates while the context is fresh. A start hook can load team-specific context dynamically so every developer gets the right setup for their module without manual configuration. For automated checks like linting and formatting, hooks enforce the rules deterministically and produce more consistent results than relying on Claude to remember an instruction.

Skills keep the right expertise available on-demand without bloating every session. In a large codebase with dozens of task types, not all expertise needs to be present in every session. Skills solve this through progressive disclosure, offloading specialized workflows and domain knowledge that would otherwise compete for context space and loading them only when the task calls for it. For example, a security review skill loads when Claude is assessing code for vulnerabilities, while a document processing skill loads when a code change is made and documentation needs to be updated.

Skills can also be scoped to specific paths so they only activate in the relevant part of the codebase. A team that owns a payments service can bind their deployment skill to that directory, so it never auto-loads when someone is working elsewhere in the monorepo.